European Journal of Operational Research 317 (2024) 16–24

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/eor

Discrete Optimization

Maximizing the net present value of a project under uncertainty: Activity delays and dynamic policies

Salim Rostami [a], Stefan Creemers [a,b,*], Roel Leus [b]

[a] IESEG School of Management, University Lille, CNRS, UMR 9221 - LEM - Lille Economie Management, F-59000 Lille, France
[b] ORSTAT, KU Leuven, 3000 Leuven, Belgium

ARTICLE INFO

Keywords:
Project scheduling
Net present value
Discrete scenarios
Activity delay
Dynamic policies

ABSTRACT

We study a project with stochastic activity durations and cash flows; we model the uncertainty using discrete scenarios. The project entails precedence-related activities, each of which incurs a cash flow that may be positive (inflow) or negative (outflow). The problem is to find a scheduling policy that maximizes the expected net present value of the project. A scheduling policy decides the starting time of each activity under every possible realization of the unknown parameters. Ideally, one wants to expedite the inflows (e.g., incoming payments), while delaying the outflows (e.g., costs) as much as possible, without violating the project deadline. In this article, we devise an exact and a heuristic method to define policies within two new classes of scheduling policies. The first policy class generalizes all existing static policies in the literature and further illustrates the importance of intentional activity delays from both a theoretical as well as an empirical point of view. Whereas the literature on project scheduling has mainly focused on static policies, we also propose a second class of dynamic policies. We show that dynamic policies outperform static policies by means of extensive computational experiments.

1. Introduction

The goal of project scheduling is to determine a schedule that defines when each project activity starts and ends, while respecting the precedence and resource constraints. These schedules can be evaluated based on different criteria, such as maximizing the expected profit, minimizing the project duration (makespan), and so on. The project parameters can either be deterministic or stochastic. In a deterministic project, all parameters are fixed and fully known before project execution. Stochastic projects, however, involve a level of uncertainty or randomness in, for instance, activity duration, costs, or resource availability.

One of the earliest references on project scheduling with uncertainty is Malcolm, Rosenbloom, Clark, and Fazar (1959), who focuses on the expected makespan of the project. In fact, the objective of minimizing the expected makespan of a stochastic project has been thoroughly studied; see Ashtiani, Leus, and Aryanezhad (2011), Ballestín and Leus (2009), Chtourou and Haouari (2008), Creemers (2015), Deblaere, Demeulemeester, and Herroelen (2011), Fang, Kolisch, Wang, and Mu (2015), Li and Womer (2015), Rostami, Creemers, and Leus (2018), Stork (2001), and Van de Vonder, Demeulemeester, Herroelen, and Leus (2005). For a detailed review of the work on this problem, we refer to Bruni, Beraldi, and Guerriero (2015).

Despite its popularity, the makespan objective largely disregards the financial aspects of the project. When it comes to large and capital-intensive projects (IT, construction, R&D, manufacturing, etc.), a proper coordination of the cash flows is crucial for the financial viability of the project. For such cases, the decision maker will typically prefer to schedule the project such that the Net Present Value (NPV) is maximized.

The NPV is calculated by discounting expected future cash flows by the rate of return offered by equivalent investment alternatives in the capital market. Many companies use their weighted average cost of capital as a suitable discount rate when budgeting for a new project (Brealey, Myers, Allen, & Mohanty, 2012). The inclusion of economic considerations in project scheduling can be traced back to Kelly (1961). However, the stochastic scheduling problem with the objective to optimize a financial performance measure has not been studied as extensively as the problem that aims to minimize the expected makespan.

In this article, we focus on the maximization of the Expected NPV (ENPV) of a project and use discrete scenarios to represent stochastic activity durations and cash flows. Due to the complexity of this problem, it is often not possible to determine a globally optimal policy. As a result, the literature has focused on classes of policies that are

* Corresponding author at: IESEG School of Management, University Lille, CNRS, UMR 9221 - LEM - Lille Economie Management, F-59000 Lille, France.
E-mail address: s.creemers@ieseg.fr (S. Creemers).

https://doi.org/10.1016/j.ejor.2024.03.029
Received 19 May 2023; Accepted 19 March 2024
Available online 20 March 2024

computationally tractable. We propose two new policy classes and devise an exact and a heuristic method to search for scheduling policies. The first class of policies, called Activity Delay (AD) policies, generalizes all existing policy classes in the literature on this problem. Moreover, we further illustrate the importance of intentional activity delays from both a theoretical as well as an empirical point of view. Whereas the literature on project scheduling has mainly focused on obtaining scheduling policies that consider only project parameters that have already been realized (static policies), we also propose a second class of dynamic policies that also takes into account possible future realizations of the project. Our extensive computational experiments show that dynamic policies significantly outperform static policies.

The remainder of this article is organized as follows. In Sections 2 and 3, we review the literature and formulate the problem. Section 4 is devoted to defining and comparing different classes of scheduling policies. In Section 5, we propose an exact and a heuristic algorithm to find high-quality scheduling policies in the two aforementioned policy classes. The results of two computational experiments are discussed in Section 6. Section 7 concludes.

2. Literature review

In the literature, we distinguish between the deterministic problem to maximize the NPV of a project, and the stochastic problem where some of the parameters are random variables. Deterministic project scheduling to maximize the NPV goes back to Russell (1970). Russel uses an activity-on-node graph representation of projects, which means that a graph $G(N, A)$ is part of the problem input, with $N = \{1, \dots, n\}$ the set of activities and A the set of transitive precedence constraints. Each ordered pair $(i, j) \in A$ implies that activity i should be finished before j can be started. More specifically, A is a strict partial order on N, i.e., it is irreflexive (pairs $(j, j) \notin A$), asymmetric (if $(i, j) \in A$, then $(j, i) \notin A$), and transitive (if $(i, j), (j, k) \in A$, then $(i, k) \in A$). Each activity $j \in N$ has a known duration $d_j \geq 0$ and a known cash flow c_j, which may be positive (inflow) or negative (outflow). Activities 1 and n are the unique source and sink of the project network, respectively, i.e., $\forall j \in N \setminus \{1\} : (1, j) \in A$ and $\forall j \in N \setminus \{n\} : (j, n) \in A$. We have $d_1 = d_n = 0$, $c_1 = 0$, and c_n is the final payment of the project. The cash flows are incurred at the start of each activity and are discounted using a known discount factor β, which is the present value of a cash flow with value $+1$ after one time period (e.g., one year); we assume that $0 < \beta < 1$.

The problem can now be stated as follows:

$$\max \quad \sum_{j \in N} c_j \beta^{x_j} \tag{2.1}$$

$$\text{subject to} \quad x_j \geq x_i + d_i, \qquad \forall (i, j) \in A, \tag{2.2}$$

$$x_1 = 0, \tag{2.3}$$

$$x_j \geq 0, \qquad \forall j \in N. \tag{2.4}$$

The decision variable x_j determines the starting time of activity j. The constraints ensure that the precedence constraints are satisfied and that the starting times are non-negative. A project deadline δ can be enforced by including $x_n \leq \delta$. Russell (1970) tackles the problem as a sequence of linear programs whose objective functions are obtained using linearization around the current candidate solution. The duals of these approximations can be solved as network-flow problems.

Grinold (1972) substitutes β^{x_j} by y_i to linearize Russel's formulation:

$$\max \quad \sum_{j \in N} c_j y_j \tag{2.5}$$

$$\text{subject to} \quad y_j \leq y_i \beta^{d_i}, \qquad \forall (i, j) \in A, \tag{2.6}$$

$$y_1 = 1, \tag{2.7}$$

$$y_j \geq 0, \qquad \forall j \in N. \tag{2.8}$$

A constraint for a project deadline (denoted δ) can be included as $y_n \geq \beta^{\delta}$. Grinold solves this problem in polynomial time using a variant of the network simplex algorithm. Neumann and Zimmermann (2000) and Schwindt and Zimmermann (2001) propose other methods that prove to be more efficient than Grinold's algorithm in practice. For a survey of related work, we refer to Demeulemeester and Herroelen (2006).

Stochastic versions of this problem are generally perceived to be computationally difficult to tackle. This is mainly due to the fact that the objective function is non regular and the timing of observing the realized values of the random variables (pertaining to activity durations and cash flows) depends on the scheduling decisions, that is, the information structure of the problem is decision dependent. Problems of this type are typically very challenging to solve, and they have received fairly limited attention in the stochastic programming literature (see also Wiesemann and Kuhn (2015)). A solution to a stochastic max-NPV problem is a scheduling policy that decides the starting time of each activity under every possible realization of the unknown parameters. An optimal policy expedites the inflows (e.g., incoming payments), while delaying the outflows (e.g., costs) as much as possible, without violating the precedence constraints and the project deadline constraint.

The literature on stochastic project scheduling to maximize the ENPV is rather scarce. Buss and Rosenblatt (1997), Creemers, Leus, and Lambrecht (2010), Creemers (2018a), and Hermans and Leus (2018) consider exponentially distributed activity durations and employ continuous-time Markov chains to obtain optimal scheduling policies. For activities that have generally distributed activity durations, Creemers (2018b) finds closed-form approximations for the distribution of the NPV of a serial project. He also addresses the problem of finding the maximum ENPV of a serial project and shows that it is equivalent to the least cost fault detection problem. Buss and Rosenblatt (1997) propose an exact method to determine the optimal delay of two parallel activities with cash outflows. They also develop approximate procedures for larger instances. Benati (2006) and Wiesemann, Kuhn, and Rustem (2010) work with discrete random variables (discrete scenarios). Benati (2006) proposes a heuristic method to search within a new class of static scheduling policies. Benati's policies assign an intentional delay to each activity. In every scenario, each activity starts immediately after its assigned delay following the finishing time of its predecessors. Wiesemann et al. (2010) propose a new class of static policies that assigns an earliest possible starting time to each activity, referred to as a "Target Processing Time" (TPT). In every scenario, each activity starts either at its TPT or at the latest completion time of its predecessors, whichever occurs the latest. They develop a branch-and-bound algorithm to optimize within this class of policies. For a survey on max-ENPV stochastic project scheduling, we refer to Herroelen, Van Dommelen, and Demeulemeester (1995) and Wiesemann and Kuhn (2015).

3. Problem definition

Unlike the deterministic case defined in Section 2, we define activity durations $\tilde{\mathbf{d}} = (\tilde{d}_j)_{j \in N}$ and cash flows $\tilde{\mathbf{c}} = (\tilde{c}_j)_{j \in N}$ to be random variables with known probability distributions. In addition, we assume these distributions to be discrete. Without loss of generality, we assume that all cash flows are incurred at the start of the activities.

A policy π maps a scenario $s = (\mathbf{d}^s, \mathbf{c}^s)$ to starting times $(\pi(j|s))_{j \in N}$, with $\mathbf{d}^s = (d_j^s)_{j \in N}$ and $\mathbf{c}^s = (c_j^s)_{j \in N}$ being the realizations of random variables $\tilde{\mathbf{d}}$ and $\tilde{\mathbf{c}}$ in scenario s. A policy π is called feasible if (1) it is non-anticipative, (2) for any realizations $\mathbf{d}$ and $\mathbf{c}$, the activity starting times are precedence-feasible, and (3) the project is finished by deadline δ. The non-anticipativity constraint states that when making a decision, the project manager cannot use information that becomes available after the decision moment. More formally, any two realizations $s = (\mathbf{d}^s, \mathbf{c}^s)$ and $s' = (\mathbf{d}^{s'}, \mathbf{c}^{s'})$ should lead to $\pi(j|s) = \pi(j|s')$,

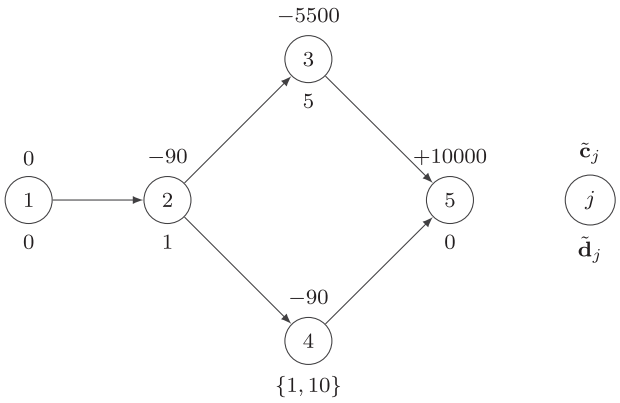

Fig. 1. Example 1.

for all $j \in N$, as long as $d_i^s = d_i^{s'}$ and $c_i^s = c_i^{s'}$ for any $i \in N$ with $\pi(i|s) + d_i^s \leq \pi(j|s)$.

The problem is to find a scheduling policy π that maximizes the ENPV of the project, over the set S of all possible scenarios. Let Π be a set of finitely many static policies. All static policies considered in this article are non-anticipative by definition, so we assume the policies in Π to be non-anticipative as well. The realization probability of each scenario $s \in S$ is denoted by p_s, with $\sum_{s\in S} p_s = 1$ and $p_s = 1/|S|$. Working with a discount factor $0 < \beta < 1$, we propose the following conceptual formulation (CF) for the project instance (N, A, S, β).

$$\max \quad G(\pi) = \sum_{s\in S} p_s \sum_{j\in N} c_j^s \beta^{\pi(j|s)} \tag{3.9}$$

$$\text{s.t.} \quad \pi(j|s) \geq \pi(i|s) + d_i^s, \qquad \forall (i,j) \in A,\ \forall s \in S, \tag{3.10}$$

$$\pi(n|s) \leq \delta, \qquad \forall s \in S, \tag{3.11}$$

$$\pi(j|s) \geq 0, \qquad \forall j \in N,\ \forall s \in S. \tag{3.12}$$

$$\pi \text{ is non-anticipative.} \tag{3.13}$$

Example 1. Consider the example project depicted in Fig. 1 with deterministic cash flows and uncertain durations. There are two scenarios, with $\mathbf{c}^1 = \mathbf{c}^2 = (0, -90, -5\,500, -90, +10\,000)$, $\mathbf{d}^1 = (0, 1, 5, 1, 0)$, and $\mathbf{d}^2 = (0, 1, 5, 10, 0)$. The realization probabilities of both scenarios are $p_1 = p_2 = 0.5$. Moreover, we have $\beta = 0.9$ and $\delta = 40$. An optimal policy π^* starts activities 1, 2, and 4 as soon as possible, i.e., $\forall s \in S : \pi^*(1|s) = 0$, $\pi^*(2|s) = 0$, and $\pi^*(4|s) = 1$. Activity 3, on the other hand, is postponed for at least one time unit after the start of activity 4. If activity 4 is finished after one time unit, then activity 3 starts immediately ($\pi^*(3|1) = 2$); otherwise, it starts with a total delay of five time units after the start of activity 4 ($\pi^*(3|2) = 6$). This policy leads to an ENPV of +100.57. In other words, it is optimal to wait to start activity 3 until we know whether or not activity 4 will take 1 or 10 days.

For this example, a policy π' that has the same starting times for activities 2 and 4, but with $\pi'(3|1) = 1$ and $\pi'(3|2) = 6$, has a higher ENPV (+118.80). Such a policy, however, is anticipative since it uses information about the realized duration of activity 4, which is not available at time 1. □

4. Scheduling policies

In this section, we define existing and new classes of policies, and discuss their ability to solve on the original optimization problem.

4.1. Static policies

At any decision time t, a static policy decides to start (or delay) an eligible activity j conditioned only on the information of scenario s that has already been realized (where an activity j is eligible to start at time t if it has not yet started and if its predecessors are finished). Below, we define three existing subclasses of static policies from the scheduling literature. We also propose a new class of static policies that generalizes all subclasses available in the literature.

4.1.1. Rigid policies

A *rigid* policy prescribes scenario-independent starting times for each activity (Wiesemann et al., 2010). For a formal description of this class of policies, we simplify $\pi(j|s)$ to $\pi(j)$, and replace Constraints (3.10)–(3.12) with:

$$\pi(j) \geq \pi(i) + d_i^s, \qquad \forall (i,j) \in A, \forall s \in S, \tag{4.14}$$

$$\pi(n) \leq \delta, \tag{4.15}$$

$$\pi(j) \geq 0, \qquad \forall j \in N. \tag{4.16}$$

Rigid policies are non-anticipative due to Constraint (4.14). The resulting formulation can be linearized using Grinold's (Grinold, 1972) variable substitution. Although rigid policies are interesting from a computational point of view, they are overly conservative as the scenario-independent starting times need to remain feasible in every scenario. In other words, they take the maximum duration of each activity to make a scenario-independent schedule. As a result, in many instances, either no feasible rigid policy exists that respects the project deadline, or they lead to poor ENPV performance.

4.1.2. Target processing time policies

Wiesemann et al. (2010) propose the class of TPT policies. Each TPT policy specifies a scenario-independent release time r_j for each activity j.

Working with this class of policies, we replace Constraints (3.10) and (3.12) with:

$$\pi(j|s) = \max\left\{ r_j, \max_{i:(i,j)\in A} \left\{ \pi(i|s) + d_i^s \right\} \right\}, \qquad \forall j \in N,\ \forall s \in S, \tag{4.17}$$

$$r_j \geq 0, \qquad \forall j \in N. \tag{4.18}$$

TPT policies are non-anticipative due to Constraint (4.17). This constraint results in a non-convex set of feasible solutions, however, can be relaxed such that a linear formulation is obtained (Grinold, 1972):

$$\pi(j|s) \geq \pi(i|s) + d_i^s, \qquad \forall (i,j) \in A,\ \forall s \in S, \tag{4.19}$$

$$\pi(j|s) \geq r_j, \qquad \forall j \in N,\ \forall s \in S. \tag{4.20}$$

Wiesemann et al. (2010) show that the relaxation can be solved efficiently as a deterministic max-NPV project scheduling problem. The solutions of this problem, however, may violate the non-anticipativity constraint in the presence of activities with cash outflows. More specifically, any solution (r, π) to this relaxation with a decision variable $\pi(j|s)$ that satisfies the strict inequality:

$$\pi(j|s) > \max\left\{ r_j, \max_{i:(i,j)\in A} \left\{ \pi(i|s) + d_i^s \right\} \right\}, \qquad \forall j \in N,\ \forall s \in S,$$

is anticipative. In other words, we would only delay the start of an activity j beyond its release time r_j and the completion of all its predecessors if we have information that has not yet been revealed. Wiesemann et al. (2010) develop a branch-and-bound algorithm to remove such violations and find the best TPT policy. Given an optimal anticipative solution in the root node, their branching scheme fixes the anticipative $\pi(j|s)$ to either r_j, or $\pi(i|s) + d_i^s$ for one i with $(i,j) \in A$, and every such fixation leads to a new child node.

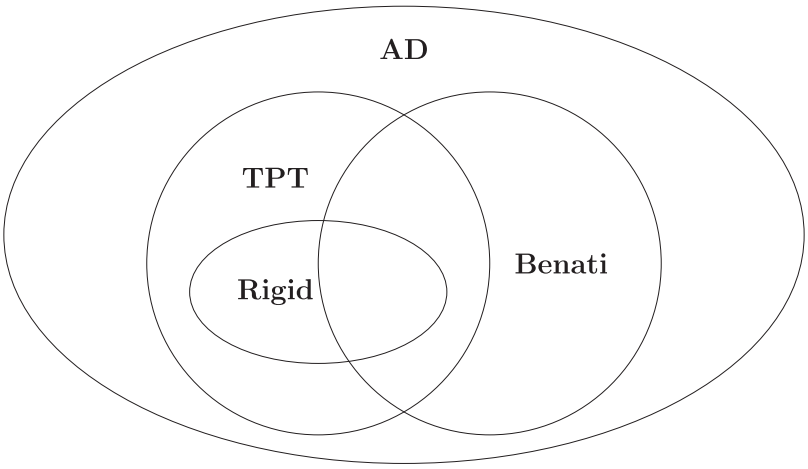

Fig. 2. Taxonomy of static policy classes.

4.1.3. Benati policies

Benati policies (Benati, 2006) try to start activities with cash outflows as late as possible by delaying them beyond the finishing time of their predecessors.

A Benati policy determines a fixed scenario-independent minimum delay b_j following the completion of all the predecessors of activity j. As such, a Benati policy imposes a minimum delay following the earliest starting time of j. Benati (2006) proposes a two-stage heuristic algorithm to search within this class of policies. Wiesemann et al. (2010) exclude Benati policies from consideration, due to their higher optimization complexity compared to TPT policies.

4.1.4. Activity delay policies

In this work, we propose the class of *Activity Delay* (AD) policies. Similar to Benati policies, AD policies are also based on delaying activities beyond the finishing time of their predecessors. We define an AD policy by means of a set of scenario-independent time lags l_{ij}, with $(i,j) \in A$. Each l_{ij} is the minimum delay between the end of activity i and the start of activity j.

For a description of AD policies, we replace Constraints (3.10) and (3.12) with:

$$\pi(j|s) = \max_{i:(i,j)\in A} \left\{\pi(i|s) + d_i^s + l_{ij}\right\}, \qquad \forall j \in N,\ \forall s \in S, \tag{4.21}$$

$$l_{ij} \geq 0, \qquad \forall (i,j) \in A. \tag{4.22}$$

AD policies are non-anticipative because of Constraint (4.21).

While Benati policies only delay activities beyond their immediate predecessors, AD policies also prescribe delays beyond each transitive predecessor. Therefore, for any activity j, AD policies assign target processing times l_{1j} and activity delays l_{ij}, with $(i,j) \in A$ and $i > 1$. In that sense, AD policies generalize both Benati and TPT policies. A formal statement of this result is provided in Section 4.1.5. In addition, note that Wiesemann et al. (2010) already coined the term "activity delay policy", however, their (informal) definition is different from our definition of an AD policy.

Unfortunately, due to the additional decision variable l_{ij} in the right-hand side of (4.21), Grinold's variable substitution (Grinold, 1972) does not resolve the nonlinearity. In Section 5.1, we present a branch-and-bound algorithm for finding the best AD policy.

4.1.5. Taxonomy of static policies

In this section, we show that AD policies generalize rigid policies, Benati policies, and TPT policies. A taxonomy of the different static policy classes is given in Fig. 2.

Proposition 1. *The class of AD policies is a superset of the class of Benati policies.*

Proof. Every Benati policy is an AD policy, where for any job $j \in N$ and any immediate predecessor $i \in N$ we have $l_{ij} = b_j$, and for any transitive predecessor $k \in N$ that is not an immediate predecessor we have $l_{kj} = 0$. □

Proposition 2. *The class of AD policies is a superset of the class of TPT policies.*

Proof. Every TPT policy is an AD policy, where for any $j \in N$, we have $l_{1j} = r_j$ and $l_{ij} = 0$ for any $i \in N \setminus \{1\}$, $(i,j) \in A$. □

Proposition 3. *The class of AD policies is a superset of the class of rigid policies.*

Proof. Suppose that $(\tau_1, \ldots, \tau_n)$ is a feasible rigid policy, where τ_j is the scenario-independent starting time of activity j. Every rigid policy is an AD policy, where for any $j \in N$, we have $l_{1j} = \tau_j$, and $l_{ij} = 0$ for any $i \in N \setminus \{1\}$, $(i,j) \in A$. □

Proposition 4. *The class of TPT policies is a superset of the class of rigid policies.*

Proof. Suppose that $(\tau_1, \ldots, \tau_n)$ is a feasible rigid policy, where τ_j is the scenario-independent starting time of activity j. Every rigid policy is a TPT policy, where for any $j \in N$, we have $r_j = \tau_j$. □

It is straightforward to see that rigid policies are not a subset of Benati policies, and that Benati policies are not a subset of TPT policies (and vice versa). Even though AD policies generalize all existing static policies, it is not difficult to come up with an example of a static policy that is not an AD policy (e.g., consider a policy that starts activities with a delay that depends on the percentage complete of the project). In addition, AD policies (and therefore also rigid, Benati, and TPT policies) use delays that have been predetermined before the start of the project. This, however, need not be the case. Static policies may also use heuristic rules that determine the delay of an activity based on project parameters that have already been realized at the decision time.

Example 2. Consider the example depicted in Fig. 3, with $p_1 = p_2 = 0.5$, $\beta = 0.9$, and $\delta = 40$ (same as in Example 1). We can show that the best TPT policy π^*_{TPT} with $\mathbf{r} = (0,0,5,0)$ leads to $G(\pi^*_{\text{TPT}}) = +12.43$. On the other hand, the best Benati policy π^*_{B}, with $b_2 = b_3 = b_4 = b_5 = 0$, has $G(\pi^*_{\text{B}}) = +10.89$. In this example, π^*_{TPT} is the best AD policy as well. □

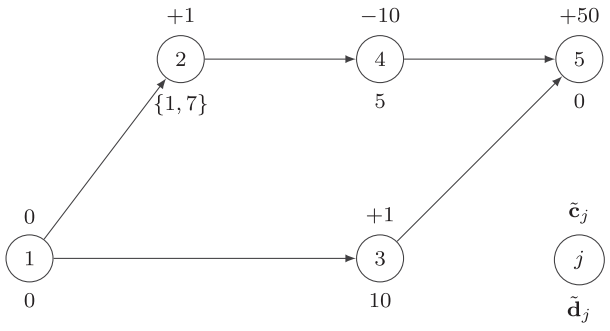

Fig. 3. Example 2.

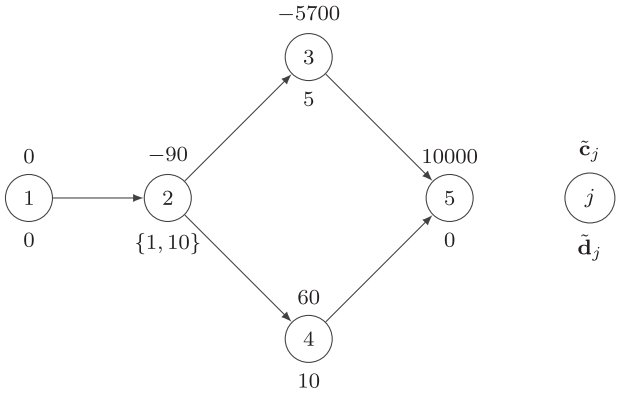

Fig. 4. Example 3.

Example 3. Consider the example depicted in Fig. 4, with $p_1 = p_2 = 0.5$, $\beta = 0.9$, and $\delta = 40$ (same as in Example 1). We can show that the best TPT policy π^*_{TPT} with $\mathbf{r} = (0, 20, 35, 0, 0)$ obtains $G(\pi^*_{\text{TPT}}) = -1.26$ (i.e., the project is not profitable when using TPT policies). On the other hand, the best AD policy π^*_{AD} with $l_{1,2} = 0$, $l_{1,3} = 0$, $l_{2,3} = 5$, and $l_{1,4} = l_{2,4} = 0$ has $G(\pi^*_{\text{AD}}) = +23.00$. Note that in this example, π^*_{TPT} completes the project as late as possible. Therefore, decreasing the project deadline decreases the ENPV of any TPT policy. For $\delta = 20$, for instance, the best TPT policy has $\mathbf{r} = (0, 0, 15, 0, 0)$ and leads to an ENPV of -10.35. □

Note that, because $G(\pi^*_{\text{AD}}) \geq G(\pi^*_{\text{TPT}})$, the difference between $G(\pi^*_{\text{AD}})$ and $G(\pi^*_{\text{TPT}})$ can be made arbitrarily high by scaling cost parameters. In addition, note that the class of AD policies does not necessarily contain a globally optimal policy. In Example 1, for instance, the best TPT and AD policies both lead to an ENPV of -67, while the ENPV of the optimal policy π^* is $+100.57$.

4.2. Dynamic policies

In contrast to a static policy, a dynamic policy may also decide to start (or delay) an eligible activity j conditioned on information of active scenarios in S_t that has not yet been realized by decision time t (with S_t the subset of scenarios for which all project information is in accordance with the project parameters that are realized by time t). By looking at the remaining active scenarios at time t (or a sample thereof), dynamic policies determine (or approximate) the expected value of starting (or delaying) an activity j. Since dynamic policies do not know which of the active scenarios will eventually be realized (they only know that each active scenario will be realized with a given probability), they are not anticipative. For an example of dynamic policies in the project scheduling literature, we refer to Li and Womer (2015).

In Example 1, π^* (that has an ENPV equal to $+100.57$) decides the start of activity 3 at time $t = 2$ based on the remaining active scenario (i.e., either activity 4 is finished or it is still ongoing). As a result, π^* is a dynamic policy. Dynamic policies (as well as static policies) can be represented by decision trees. Fig. 5, for instance, depicts the decision tree associated with optimal policy π^*, where at any time t, F_t and O_t denote the sets of finished and ongoing activities, respectively.

As dynamic policies generalize static policies, the size of the search space for finding the best dynamic policy is larger than the size of the search space for finding the best static policy. Therefore, for a given instance, computing an optimal dynamic policy is perceived to be more challenging than calculating an optimal static policy. However, the problem can be simplified if one uses an iterative heuristic that makes decisions (from a limited set of decisions) each time new information becomes available. For instance, at each time t during the execution of a project, we need only a partial policy to decide the start of the eligible activities at t. In Section 5.2, we propose a heuristic approach that iteratively optimizes partial dynamic policies for the remainder of the project and its potential scenarios.

5. Solution methods

In this section, we first propose an exact method for finding an optimal AD policy (Section 5.1). Next, we present a heuristic method for finding a dynamic policy (Section 5.2). Later in Section 6, we compare the performance of these approaches with the state-of-the-art algorithm of Wiesemann et al. (2010).

5.1. Optimal AD policies

The objective function of CF (see Section 3) can be linearized if written as a so-called time-indexed formulation (TF). Without loss of generality, let $T = \{0, 1, \ldots, \delta\}$ be the set of all the decision times (see also infra) for a given project. The decision variable $\pi(j, t|s) = 1$ if in scenario s, activity j is started at time $t \in T$, and $\pi(j, t|s) = 0$ otherwise.

$$\max \quad \sum_{s\in S} p_s \sum_{j\in N} \sum_{t\in T} c_j^s \beta^t \pi(j,t|s) \tag{5.23}$$

$$\text{s.t.} \quad \pi(1,0|s) = 1, \qquad \forall s \in S, \tag{5.24}$$

$$\sum_{t\in T} t\pi(j,t|s) = \max_{i:(i,j)\in A} \left\{ \sum_{t\in T} t\pi(i,t|s) + d_i^s + l_{ij} \right\}, \qquad \forall j \in N \setminus \{1\}, \forall s \in S, \tag{5.25}$$

$$\sum_{t\in T} \pi(j,t|s) = 1, \qquad \forall j \in N, \forall s \in S, \tag{5.26}$$

$$l_{ij} \geq 0 \qquad \forall (i,j) \in A, \tag{5.27}$$

$$\pi(j,t|s) \in \{0,1\} \qquad \forall j \in N, t \in T, \forall s \in S. \tag{5.28}$$

Following Wiesemann et al. (2010), we can linearize TF by relaxing Equality (5.25) to greater-than-or-equal constraints. We will refer to this relaxation as RTF. Note that if a solution (b, π) to RTF contains a $\pi(j, t|s)$ that satisfies the strict inequality:

$$\sum_{t\in T} t\pi(j,t|s) > \max_{i:(i,j)\in A} \left\{ \sum_{t\in T} t\pi(i,t|s) + d_i^s + l_{ij} \right\},$$

the non-anticipativity constraint is violated. We propose a branch-and-bound procedure, similar to the algorithm of Wiesemann et al. (2010), to remove such violations and to find an optimal AD policy. Given an anticipative solution in the root node, the branching scheme fixes the starting time of an anticipative activity j to the finishing time of an activity i with $(i, j) \in A$. Every such fixation leads to a new child node. Note that, whereas Wiesemann et al. (2010) solve an LP in each node, we solve an IP.

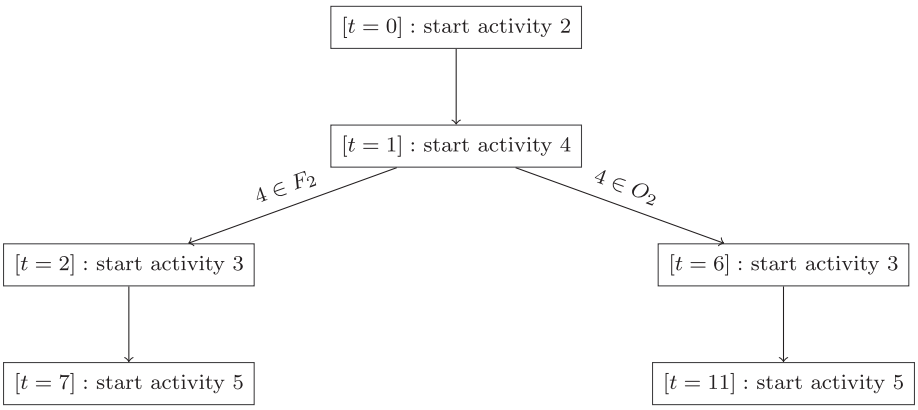

Fig. 5. The decision tree of policy π^* in Example 1.

5.2. Dynamic policies: A heuristic method

Obtaining an optimal dynamic policy may require involved calculations to determine optimal decisions at each decision time (e.g., by using dynamic programming or branch-and-bound methods similar to those discussed in Section 5.1). Such a procedure can quickly become computationally intractable. To address this issue, in this section, we propose a fast heuristic approach that, at each decision time during the project execution, devises a *partial* dynamic policy that only takes into account the eligible activities.

Firstly, at each point in time t, let E_t be the set of eligible activities. An activity $j \in N$ is eligible to start at time t if it has not yet started and if its predecessors are finished, i.e., if $j \notin F_t \cup O_t$ and $i \in F_t, \forall i : (i,j) \in A$.

Secondly, we define a decision time as a point in time when new information regarding the project becomes available. More precisely, decision times are: (1) when an activity is finished, and its realized duration is known, (2) when the duration of an ongoing job j surpasses a potential value d_j^s in at least one scenario $s \in S$, and (3) when there is a possibility of missing the project deadline δ if an eligible activity does not start immediately (i.e., if time t is the latest starting time of an eligible activity $j \in E_t$ in at least one scenario $s \in S_t$, we may have to decide to start j at time t).

Thirdly, in our heuristic approach, we use an upper bound inspired by the perfect information relaxation that is used mainly in stochastic programming (see, for instance, Boncompte (2018), Huang, Vertinsky, and Ziemba (1977), Santos, Poss, and Nace (2018)). The Expected Value given Perfect Information (EV|PI) upper bound assumes "perfect information" on activity durations and cash flows. In other words, we assume that we know the specific scenario that will unfold before project execution. For a given scenario s, the V|PI can be obtained by solving the deterministic problem (N, A, s, β). Therefore, the EV|PI of a problem instance (N, A, S, β) is equal to $\sum_{s \in S} p_s G^*(N, A, s, \beta)$, where $G^*(N, A, s, \beta)$ is the NPV of deterministic problem (N, A, S, β) if optimal decisions are made. Using Grinold's variable substitution and formulation, $G^*(N, A, s, \beta)$ can be calculated efficiently (Grinold, 1972).

At each decision time t, and for each eligible activity j, we can either start j immediately, or wait for more information to become available (at a later time). In order to decide, we compare the EV|PI of the project in two cases: (1) when j is started at t, and (2) when j is started at the next decision time $t' > t$. Assuming discrete scenarios, the earliest next decision time t' can easily be identified (see also Example 4).

Example 4. In Example 1, assume that at $t = 1$, activity 4 is ongoing, and activity 3 is eligible to start: $F_1 = \{1,2\}$, $O_1 = \{4\}$, $E_1 = \{3\}$, and $S_1 = \{1,2\}$. In this case, $t = 2$ will be a decision time, where we find out whether $\tilde{d}_4$ equals 1 or 10. At $t = 1$, we need to decide whether to start activity 3 together with 4, or to wait for more information at $t = 2$. To do so, we calculate the EV|PI $\sum_{k \in S_1} G^*(N, A, k, \beta)$ for the following two cases:

1. We let activity 3 start at $t = 1$ in both scenarios ($\forall s \in S_1 : \pi(3|s) = 1$).
2. We postpone activity 3 in both scenarios ($\forall s \in S_1 : \pi(3|s) \geq 2$).

In both cases, we force the finished and ongoing activities to start at their previously decided starting times, i.e., $\forall s \in S_1 : \pi(1|s) = \pi(2|s) = 0$, and $\pi(4|s) = 1$. In this example, Case 2 (with $\pi(3|s) \geq 2$) leads to an EV|PI of +100.57, while Case 1 (with $\pi(3|s) = 1$) has an EV|PI of −894.74. Consequently, at $t = 1$ we decide to postpone activity 3. For this example, our heuristic approach leads to the optimal policy π^* described in Example 1. □

Our iterative heuristic approach initializes at $t = 0$ with $F_0 = \{1\}$, $O_0 = \emptyset$, and $S_0 = S$. At each decision time $0 \leq t \leq \delta$, we obtain the EV|PI of the project, while forcing the finished and ongoing activities to start at their previously decided starting times. Together with the EV|PI, we also obtain new starting times $\pi(j|s)$ for each $j \in N \setminus (F_t \cup O_t)$, and $s \in S_t$. Three following cases are possible. (1) If $\nexists j \in E_t, s \in S_t$ such that $\pi(j|s) = t$, then we do not start any of the eligible activities and proceed to the next decision time. (2) If $\exists j \in E_t$ for which $\forall s \in S_t$ we have $\pi(j|s) = t$, then we start j immediately. (3) If $\exists j \in E_t$, $s, s' \in S_t$ such that $\pi(j|s) = t$ and $\pi(j|s') \neq t$, then the non-anticipativity constraint is violated. To resolve the violation, we consider the two following options:

1. We start j at t in all scenarios, i.e., $\forall s \in S_t$, we let $\pi(j|s) = t$.
2. We postpone j in all the scenarios in $\{s \in S_t : \pi(j|s) > t\}$ until the next decision time.

For each option, we calculate the EV|PI bound, with the same procedure as explained above. Finally, we choose the option that has the highest EV|PI, and then go to the next decision time.

Proposition 5. *Forcing option 1 or 2 does not empty the solution space in any scenario.*

Proof. We know that until t, the information that is revealed from the scenarios in S_t has been identical. Therefore, based on the non-anticipativity constraint, the sets of previous decisions that are made before t are also the same. Consequently, if $\exists s' \in S_t$, where activity j can be started at $t' \geq t$, then $\forall s \in S_t$, there is at least one feasible schedule in which $\pi(j|s) = t'$. □

To ensure that the project finishes before the deadline, the activities are forced to start not later than their latest starting time. At each time t, we start the eligible activity $j \in E_t$, if the next decision time is later than the latest starting time of j in at least one scenario $s \in S_t$.

If $O_t = \emptyset$ then the next decision times are the latest starting times of the eligible activities in E_t. In this case, postponing all the eligible activities to the next decision time intentionally delays the end of the

project. Such a project postponement can be reasonable only if the EV|PI of the rest of the project is negative (i.e., we "abandon" the project only if we no longer expect to make a profit). Therefore, if $O_t = \emptyset$ and the total EV|PI is positive, then at least one eligible activity should be started. In this case, we start the eligible activity with the highest EV|PI if started now (option 1 above), even if it has a higher EV|PI if postponed (option 2 above).

This heuristic approach repeats the procedure described above for all scenarios in S until all project activities are processed. The heuristic progressively decides the next activities (if any) to start at every decision point. An alternative approach could, for instance, use Monte Carlo simulation to determine the value that corresponds to each decision (see, for instance, Rostami et al. (2018)). There are two reasons, however, why we opt for EV|PI. First, because we can use Grinold's method to calculate the EV|PI efficiently; calculating the EV|PI of a decision requires less computational effort than using Monte Carlo simulation. Second, the EV|PI is an exact upper bound on the value of a decision. As such, it does not suffer from the uncertainty that is inherent to Monte Carlo simulation and it allows to differentiate between two decisions that have a similar (simulated) value.

6. Computational results

In this section we present two computational experiments. In a first experiment, we compare the performance of existing static policies as well as AD policies and our heuristic dynamic policy. In this experiment, we use a set of instances (denoted Ω_1 and referred to as the "original" set) that replicates the instances of Wiesemann and Kuhn (2015) as closely as possible. Instances in Ω_1, however, only have a limited number of scenarios and allow policies to "peek into the future" (i.e., at a given time t, by looking at the realized activity durations and cash flows, we can figure out what scenarios are still active, and can use this information to better schedule activities). To eliminate the impact of "peeking into the future", we perform a second experiment in which we use a different set of instances (denoted Ω_2 and referred to as the "no-peeking" set).

For both sets of instances, we choose a random project deadline $\delta = \lambda \max_{s\in S} \Delta^s$, where Δ^s denotes the minimum makespan for scenario $s \in S$, and λ is sampled from a uniform distribution on $[1.5, 2]$. The described generation procedure ensures that feasible TPT, AD, and Rigid policies exist for all instances. Throughout this section, we employ a discount factor $\beta = 0.9675$ (following Wiesemann et al. (2010)).

All the tests have been done on the same machine with 8 GB of RAM, and an Intel(R) Core(TM) i5-8350U CPU (1.70 GHz to 1.90 GHz). The algorithms have been coded in C++. We use CPLEX Studio 12.8 to solve the (integer) linear programs.

6.1. Results: Original set of instances

The instances of set Ω_1 are constructed following the convention used in Wiesemann et al. (2010). We refer to this set as the "original" dataset. In Ω_1, activity durations and cash flows are modeled using discrete scenarios. For every scenario, the activity cash flows are sampled from a uniform distribution on $[-100, 100]$, while the durations are selected from a uniform distribution with support $[1, 10]$. In contrast to Wiesemann et al. (2010), we do not use ProGen to generate precedence networks, but build them ourselves by randomly generating arcs. For each $i, j \in N$, $(j, i) \notin A$, we add (i, j) and its transitive arcs to A with a probability of 0.2.

We use the instances of Ω_1 to compare the performance of the policy classes presented in this paper. First, we calculate the gap percentage between the objective value of each policy and the perfect information bound (EV|PI).

Table 1 presents the average gap percentages for the rigid, TPT, AD, and dynamic policies. The gap is calculated as $100 * (\text{EV|PI} - X)/\text{EV|PI}$, where X is the ENPV of the studied policy (rigid, TPT, AD, or DYN). The EV|PI is $\sum_{s\in S} p_s G^*(N, A, s, \beta)$, where $G^*(N, A, s, \beta)$ is the optimal NPV of the deterministic project with scenario s. As a result, EV|PI presents an effective upper bound on the ENPV of the project. The best rigid, TPT, and AD policies are obtained as explained in Section 5.1. DYN denotes the best dynamic policy found by our heuristic approach as defined in Section 5.2. Each setting is characterized by pair $(n, |S|)$, where $(n - 2) \in \{5, 10, 15, 20, 25, 30\}$, and $|S| \in \{2, 5, 10\}$. For each setting, ten random instances are generated. The results for each setting and each method are reported only if all the ten instances are solved. A time limit of 600 s (10 min) is reached for all the unsolved instances.

Table 1
Percentage gap between the ENPV of different policies and the EV|PI (in percent) for dataset Ω_1 (note that the EV|PI is an upper bound on the ENPV of a globally optimal policy).

$(n-2)$	$\lvert S\rvert$	rigid	TPT	AD	DYN
5	2	102.55	83.43	80.21	22.16
	5	86.30	74.90	65.17	14.60
	10	127.00	117.33	93.16	17.90
10	2	54.40	45.50	45.13	9.10
	5	176.30	89.70	68.54	4.30
	10	89.00	57.31		5.50
15	2	58.30	36.50	15.00	1.70
	5	94.10	66.00		1.12
	10	87.50			2.90
20	2	43.60	32.10	16.49	0.23
	5	84.70	48.12		1.50
	10	85.90			4.20
25	2	123.40	89.94		0.20
	5	75.10			2.30
	10	80.00			0.70
30	2	57.50			0.07
	5	64.80			0.30
	10	77.40			0.50

As shown in the table, compared to the other policies, dynamic policies perform significantly better (i.e., they lead to the highest ENPV). Moreover, compared to the branch-and-bound algorithms, the heuristic can solve larger instances with more scenarios. The AD policies noticeably improve the ENPV of the projects compared to TPT and rigid policies. The gap gets larger in projects with more activities and more scenarios. However, the performance of the exact method is limited to 25 activities or 10 scenarios.

Table 2 presents the average gap percentages between each policy class and the TPT policies (the state-of-the-art policy class). The gap is calculated as $100\times(X - \text{TPT}^*)/\text{TPT}^*$, where X is the ENPV of the studied policy (rigid, AD, or DYN), and TPT^* is the ENPV of the best TPT policy. Compared to TPT policies, AD policies increase the ENPV of the random projects by up to 27%. The ENPV improvement of dynamic policies is significantly higher than AD policies (up to 310%). The difference increases as the number of scenarios increases.

Table 3 compares the runtime of the exact and heuristic algorithms. Based on the table, our heuristic method is significantly faster than the exact methods for TPT and AD policies. The heuristic runs efficiently because, at each decision time, it solves a polynomial problem for each eligible activity and each scenario. The results for each setting and each method are reported only if all the ten instances are solved.

6.2. Results: No-peeking set of instances

The instances in set Ω_1 only consider a limited number of scenarios. As a result, policies may benefit from "peeking into the future". For instance, in Example 1 at time $t = 2$ only one active scenario remains and can decide the optimal starting time of activity 3 based on this scenario. If we want to counter the effect of "peeking into the future", we could generate all possible scenarios for each instance (rather than

Table 2
Percentage gap between the ENPV of different policies and the ENPV of the TPT policy (in percent) for dataset Ω_1.

$(n-2)$	$\|S\|$	rigid	AD	DYN
5	2	−47.77	4.00	163.34
	5	−28.60	8.82	166.70
	10	−19.04	14.49	213.44
10	2	−12.60	2.78	82.12
	5	−170.90	23.39	308.07
	10	−87.45		286.84
15	2	−35.48	16.33	87.30
	5	−55.31		191.22
20	2	−19.64	27.12	93.74
	5	−89.45		175.46
25	2	−39.19		146.18

Table 3
Computation times (in seconds) for different policies for dataset Ω_1.

$(n-2)$	$\|S\|$	rigid	TPT	AD	DYN
5	2	0.00	0.07	1.32	0.14
	5	0.01	4.64	25.21	0.49
	10	0.01	65.51	216.52	1.02
10	2	0.01	0.75	11.46	0.15
	5	0.01	47.73	268.31	0.44
	10	0.01	311.23		1.41
15	2	0.01	4.56	97.71	0.24
	5	0.02	134.71		0.60
	10	0.02			2.10
20	2	0.01	44.57	368.63	0.36
	5	0.02	398.12		1.21
	10	0.03			2.26
25	2	0.01	165.83		0.62
	5	0.01			1.56
	10	0.02			3.01
30	2	0.01			0.63
	5	0.01			1.85
	10	0.03			5.63

using only up to 10 scenarios as we did for instances of Ω_1). This approach, however, would result in a number of scenarios that is too large to handle even by a fast heuristic such as DYN. As an alternative, for each activity j, we propose to sample three random values: (1) from a uniform distribution with support $[1, 10]$ for the activity duration and (2) from a uniform distribution with support $[-100, 100]$ for the activity cash flow. Next, we order them, non-decreasingly, to obtain $d_j^{\min}$, d_j^{med}, $d_j^{\max}$, $c_j^{\min}$, c_j^{med}, and $c_j^{\max}$. Then, we assume that for each activity j, the pair $(\tilde{d}_j, \tilde{c}_j)$ is selected from $\{(d_j^{\min}, c_j^{\max}), (d_j^{\text{med}}, c_j^{\text{med}}), (d_j^{\max}, c_j^{\min})\}$. While each activity is independent from the other activities, the duration and the cost of a given activity are positively correlated (longer duration implies higher cost). Not only does this assumption make practical sense, it also limits the number of scenarios that need to be generated for each project. By considering only 3 pairs, we generate $|S| = 3^n$ scenarios for each project, allowing us to analyze projects that have up to 12 non-dummy activities (resulting in 531,441 scenarios; which is still tractable for our heuristic dynamic policy). More important, however, by considering all permutations of the 3 pairs, we eliminate the impact of "peeking into the future"; the set of active scenarios does not reveal any information on the duration and/or cash flow of the remaining activities (each pair is equally likely in the remaining active scenarios).

Using the aforementioned approach, we generate the instances of set Ω_2. For each value of $n \in \{5, 7, 10, 12\}$ we generate 10 random instances and use them to investigate the impact on the performance of rigid policies and our heuristic dynamic policy. Unfortunately, the branch-and-bound algorithms for TPT and AD policies cannot solve these instances.

Table 4
Percentage gap between the ENPV of different policies and the EV|PI for dataset Ω_2 (note that the EV|PI is an upper bound on the ENPV of a globally optimal policy).

$(n-2)$	rigid	DYN
5	19.34	4.46
7	33.81	8.54
10	54.56	15.97
12	102.30	22.68

Table 4 compares the performance of the rigid and dynamic policies. It presents the average gap percentage between each policy and the perfect information bound (EV|PI). For each size $(n-2) \in \{5, 7, 10, 12\}$, fifty random instances are generated. A time limit of 3600 s (1 h) is reached in all the unsolved instances of this section.

Based on the table, we conclude that dynamic policies still perform significantly better than rigid policies (i.e., they lead to a higher ENPV). However, compared to the first experiment (Table 1), the difference between rigid and DYN is less outspoken. This illustrates that DYN is able to benefit more from "peeking into the future" than rigid policies (as expected). Our results also shows that eliminating the effect of "peeking into the future" in fact improves the performance of rigid policies and DYN. For instances of Ω_1 (the "original" dataset that allows "peeking into the future"), DYN has gaps of up to 22.16% and 9.10% when compared to the EV|PI bound in case of $|S| = 2$ and for 5 and 10 non-dummy activities, respectively. If $|S|$ increases (i.e., if "peeking into the future" becomes more difficult), the gap reduces. In Ω_2 (the "no-peeking" set), the gap further reduces to 4.46% (for 5 non-dummy activities) and 15.97% (for 10 non-dummy activities). This illustrates that, even without "peeking into the future", DYN has consistent/excellent performance.

7. Conclusions

In this work, we have addressed the problem of maximizing the expected Net Present Value (ENPV) of a project when activity durations and cash flows are stochastic and described by discrete scenarios. We have demonstrated the importance of delaying the activities with outflows from both theoretical and empirical points of view. Theoretically, we have shown that activity delay (AD) policies can be arbitrarily better than the existing classes of policies. We have proved that the class of AD policies is a superset of the class of target processing time (TPT) policies (the current state of the art), Benati policies, and rigid policies. We have proposed a time-indexed formulation and a branch-and-bound algorithm to find an optimal AD policy. Our computational tests show that the AD policies lead to higher ENPV compared to TPT and rigid policies, but this comes at a computational cost.

Moreover, we have considered the larger class of dynamic policies that also allows to delay activities based on possible future realizations of project parameters. We have proposed a heuristic method to search within dynamic policies. Our computational results indicate that dynamic policies lead to significantly higher ENPV, when compared to static TPT and AD policies. We conclude that our heuristic approach for finding a dynamic policy is considerably faster than the other methods.

Finally, we evaluate our approaches on instances that eliminate the benefit of anticipating the future. While finding optimal TPT and AD policies for these instances is not computationally tractable, we show that our dynamic policies are still efficient and significantly outperform the rigid policies.

As a future research direction, it would be interesting to develop heuristic procedures to find good AD policies. It would also be interesting to study the performance of heuristic dynamic policies in projects with continuous stochastic durations. Another interesting topic would be to study hybrid policies, where one can for instance revise a static policy only once during the execution of the project. Other than investigating the impact of such revisions, determining the timing of a revision can be interesting from an optimization point of view.

CRediT authorship contribution statement

Salim Rostami: Conceptualization, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft, Writing – review & editing. **Stefan Creemers:** Conceptualization, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Supervision, Validation, Writing – original draft, Writing – review & editing. **Roel Leus:** Conceptualization, Formal analysis, Funding acquisition, Investigation, Methodology, Project administration, Resources, Supervision, Validation, Writing – original draft, Writing – review & editing.

References

Ashtiani, B., Leus, R., & Aryanezhad, M. (2011). New competitive results for the stochastic resource-constrained project scheduling problem: Exploring the benefits of pre-processing. *Journal of Scheduling*, *14*(2), 157–171.

Ballestín, F., & Leus, R. (2009). Resource-constrained project scheduling for timely project completion with stochastic activity durations. *Production and Operations Management*, *18*, 459–474.

Benati, S. (2006). An optimization model for stochastic project networks with cash flows. *Computational Management Science*, *3*(4), 271–284.

Boncompte, M. (2018). The expected value of perfect information in unrepeatable decision-making. *Decision Support Systems*, *110*, 11–19.

Brealey, R. A., Myers, S. C., Allen, F., & Mohanty, P. (2012). *Principles of corporate finance*. Tata McGraw-Hill Education.

Bruni, M. E., Beraldi, P., & Guerriero, F. (2015). The stochastic resource-constrained project scheduling problem. In *Handbook on project management and scheduling*: *vol. 2*, (pp. 811–835). Springer.

Buss, A., & Rosenblatt, M. (1997). Activity delay in stochastic project networks. *Operations Research*, *45*(1), 126–139.

Chtourou, H., & Haouari, M. (2008). A two-stage-priority-rule-based algorithm for robust resource-constrained project scheduling. *Computers & Industrial Engineering*, *55*, 183–194.

Creemers, S. (2015). Minimizing the expected makespan of a project with stochastic activity durations under resource constraints. *Journal of Scheduling*, *18*(3), 263–273.

Creemers, S. (2018a). Maximizing the expected net present value of a project with phase-type distributed activity durations: An efficient globally optimal solution procedure. *European Journal of Operational Research*, *267*(1), 16–22.

Creemers, S. (2018b). Moments and distribution of the net present value of a serial project. *European Journal of Operational Research*, *267*(3), 835–848.

Creemers, S., Leus, R., & Lambrecht, M. (2010). Scheduling Markovian PERT networks to maximize the net present value. *Operations Research Letters*, *38*(1), 51–56.

Deblaere, F., Demeulemeester, E., & Herroelen, W. (2011). Proactive policies for the stochastic resource-constrained project scheduling problem. *European Journal of Operational Research*, *214*(2), 308–316.

Demeulemeester, E. L., & Herroelen, W. S. (2006). *Project scheduling: A research handbook*: *vol. 49*, Springer Science & Business Media.

Fang, C., Kolisch, R., Wang, L., & Mu, C. (2015). An estimation of distribution algorithm and new computational results for the stochastic resource-constrained project scheduling problem. *Flexible Services and Manufacturing Journal*, *27*(4), 585–605.

Grinold, R. C. (1972). The payment scheduling problem. *Naval Research Logistics*, *19*(1), 123–136.

Hermans, B., & Leus, R. (2018). Scheduling Markovian PERT networks to maximize the net present value: New results. *Operations Research Letters*, *46*(2), 240–244.

Herroelen, W., Van Dommelen, P., & Demeulemeester, E. (1995). Project network models with discounted cash flows. a guided tour through recent developments. *European Journal of Operational Research*.

Huang, C. C., Vertinsky, I., & Ziemba, W. T. (1977). Sharp bounds on the value of perfect information. *Operations Research*, *25*(1), 128–139.

Kelly, J. (1961). Critical path scheduling and planning: Mathematical basis. *Operations Research*, *9*(3), 296–320.

Li, H., & Womer, N. (2015). Solving stochastic resource-constrained project scheduling problems by closed-loop approximate dynamic programming. *European Journal of Operational Research*, *246*(1), 20–33.

Malcolm, D., Rosenbloom, J., Clark, C., & Fazar, W. (1959). Application of a technique for research and development program evaluation. *Operations Research*, *7*, 646–669.

Neumann, K., & Zimmermann, J. (2000). Procedures for resource leveling and net present value problems in project scheduling with general temporal and resource constraints. *European Journal of Operational Research*, *127*(2), 425–443.

Rostami, S., Creemers, S., & Leus, R. (2018). New strategies for stochastic resource-constrained project scheduling. *Journal of Scheduling*, *21*(3), 349–365.

Russell, A. H. (1970). Cash flows in networks. *Management Science*, *16*(5), 357–373.

Santos, M. C., Poss, M., & Nace, D. (2018). A perfect information lower bound for robust lot-sizing problems. *Annals of Operations Research*, *271*, 887–913.

Schwindt, C., & Zimmermann, J. (2001). A steepest ascent approach to maximizing the net present value of projects. *Mathematical Methods of Operations Research*, *53*(3), 435–450.

Stork, F. (2001). *Stochastic resource-constrained project scheduling* (Ph.D. thesis), Technische Universität Berlin.

Van de Vonder, S., Demeulemeester, E., Herroelen, W., & Leus, R. (2005). The use of buffers in project management: The trade-off between stability and makespan. *International Journal of Production Economics*, *97*, 227–240.

Wiesemann, W., & Kuhn, D. (2015). The stochastic time-constrained net present value problem. In *Handbook on project management and scheduling*: *vol. 2*, (pp. 753–780). Springer.

Wiesemann, W., Kuhn, D., & Rustem, B. (2010). Maximizing the net present value of a project under uncertainty. *European Journal of Operational Research*, *202*(2), 356–367.